

Facultad de Ingeniería
Escuela de Ingeniería Informática

Diseño e Implementación de una Prueba de Concepto para la Priorización de Vulnerabilidades en Centros de Operaciones de Seguridad

Jorge Eduardo Álvarez Alarcón

Trabajo realizado para optar al título de
Ingeniero Civil Informático

Prof. Guía: **Cristian Carrión**

Valparaíso, Octubre 2026

Certificación

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingeniero Civil Informático.

Cristian Carrión
Profesor Guía

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingeniero Civil Informático.

Nombre Profesor Informante 1
Profesor Informante

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingeniero Civil Informático.

Nombre Profesor Informante 2
Profesor Informante

Aprobado por la Escuela de Ingeniería Informática, Universidad de Valparaíso.

Declaración sobre el uso de herramientas de Inteligencia Artificial

Yo, **Jorge Eduardo Álvarez Alarcón**, autor del trabajo "*Diseño e Implementación de una Prueba de Concepto para la Priorización de Vulnerabilidades en Centros de Operaciones de Seguridad*", declaro que el contenido de este documento es original y ha sido elaborado principalmente mediante mi propio esfuerzo intelectual, en conformidad con los principios de integridad académica exigidos por la universidad.

Declaro asimismo que se han utilizado herramientas de Inteligencia Artificial, tales como *modelos de lenguaje de gran escala para asistencia de texto*, exclusivamente como apoyo en tareas auxiliares de redacción, mejora de estilo, corrección gramatical y *revisión de coherencia sintáctica en la estructuración de párrafos*. Estas herramientas no han sido empleadas para la generación autónoma de contribuciones sustantivas del trabajo, tales como el planteamiento del problema, el diseño metodológico, la implementación ni el análisis de resultados, los cuales corresponden íntegramente al autor.

Todo contenido asistido por IA ha sido revisado, validado y adaptado críticamente, garantizando su pertinencia, exactitud y originalidad. Asumo plena responsabilidad por el contenido final del documento.

Se deja constancia de que el uso de estas herramientas ha sido realizado con conocimiento de **Cristian Carrión**, profesor/a guía, en conformidad con las normativas institucionales y buenas prácticas académicas vigentes.

Diseño e Implementación de una Prueba de Concepto para la Priorización de Vulnerabilidades en Centros de Operaciones de Seguridad

Jorge Eduardo Álvarez Alarcón[a,*], Cristian Carrión[a]

[a]*Escuela de Ingeniería Informática, Universidad de Valparaíso, Valparaíso, Chile*

Resumen

Este documento presenta el diseño e implementación de una prueba de concepto (PoC) enfocada en mitigar la carga cognitiva y la ineficiencia en el triaje dentro de los SOC. Actualmente, la fragmentación de herramientas de defensa y el uso de métricas de evaluación estáticas limitan la capacidad operativa de los analistas, generando cuellos de botella en la respuesta a incidentes. Para abordar este desafío, se propone un motor de priorización centralizado basado en los principios de orquestación SOAR y un enfoque RBVM. Mediante la correlación automatizada de vulnerabilidades estructurales latentes con alertas operacionales y fuentes dinámicas de CTI, la solución prioriza riesgos organizacionales y filtra el ruido operacional. El presente trabajo detalla la fundamentación teórica, la arquitectura de la solución y la evaluación analítica del modelo propuesto en la reducción del esfuerzo de correlación manual.

Palabras clave: SOC, SOAR, RBVM, Ruido Operacional, Ciberseguridad, Triaje

Acrónimos y Siglas

- **CISA KEV:** Known Exploited Vulnerabilities catalog (Catálogo de Vulnerabilidades Explotadas Conocidas).
- **CTI:** Cyber Threat Intelligence (Inteligencia de Amenazas Cibernéticas).
- **CVSS:** Common Vulnerability Scoring System (Sistema de Puntuación de Vulnerabilidades Comunes).
- **MISP:** Malware Information Sharing Platform (Plataforma de Intercambio de Información de Malware).
- **MTTR:** Mean Time to Respond (Tiempo Medio de Respuesta).
- **PoC:** Proof of Concept (Prueba de Concepto).
- **RBVM:** Risk-Based Vulnerability Management (Gestión de Vulnerabilidades Basada en Riesgo).

*Contacto: Jorge Eduardo Álvarez Alarcón (jorge.alvareza@estudiantes.uv.cl)

- **SIEM:** Security Information and Event Management (Gestión de Información y Eventos de Seguridad).
- **SOAR:** Security Orchestration, Automation, and Response (Orquestación, Automatización y Respuesta de Seguridad).
- **SOC:** Security Operations Center (Centro de Operaciones de Seguridad).
- **SSVC:** Stakeholder-Specific Vulnerability Categorization (Categorización de Vulnerabilidades Específica para Partes Interesadas).

Metadata del software

Tabla 1: Información general de la Prueba de Concepto (PoC)

Descripción	**Metadata**
Arquitectura	Ingesta simulada de datos estructurados (Wazuh, OpenVAS, MISP)
Motor Lógico (Backend)	Python 3.x
Base de Datos	MongoDB (NoSQL)
Interfaz (Frontend)	React.js
Entorno de ejecución	Ejecución nativa local con bases de datos en contenedores

1. Introducción

En el escenario actual de ciberseguridad, los SOC se han constituido como infraestructuras críticas donde equipos de profesionales monitorean, detectan y responden a incidentes apoyados por tecnologías avanzadas [1]. La efectividad de estas unidades es vital para la resiliencia de las organizaciones ante amenazas que evolucionan rápidamente en complejidad y escala.

Sin embargo, la proliferación de herramientas de defensa ha generado una saturación de datos que los analistas deben procesar manualmente. Este fenómeno es causado por el alto volumen de notificaciones, una excesiva tasa de alertas no prioritarias (ruido operacional) y la existencia de silos de información en tableros desconectados.

Como respuesta a esta problemática, el presente trabajo expone el diseño y desarrollo de una PoC para una plataforma de gestión centralizada basada en los principios SOAR. La principal contribución de este proyecto es la integración de fuentes de inteligencia mediante un enfoque RBVM, automatizando la correlación de telemetría para optimizar la priorización de vulnerabilidades. Cabe destacar que el alcance de este proyecto se centra en la orquestación y priorización analítica, excluyendo la ejecución de respuestas automatizadas intrusivas en la red.

El resto de este documento se estructura de la siguiente manera: la Sección 2 presenta el marco conceptual; la Sección 3 analiza el estado del arte y trabajos relacionados; la Sección 4 define la problemática y los objetivos; la Sección 5 establece la selección tecnológica; la Sección 6 detalla el diseño algorítmico propuesto; la Sección 7 define el estudio experimental para la validación; la Sección 8 presenta las limitaciones; y finalmente, la Sección 9 expone las conclusiones y el trabajo futuro.

2. Marco Conceptual

Para fundamentar el diseño del motor de orquestación, es imperativo establecer los pilares teóricos que convergen en la solución propuesta.

2.1. Centros de Operaciones de Seguridad y Sistemas SIEM

Un SOC se define como una unidad centralizada encargada de evaluar la postura de seguridad de una organización [1]. Su núcleo tecnológico es el SIEM, el cual centraliza el almacenamiento de registros (*logs*) y aplica reglas determinísticas para identificar anomalías. Herramientas como *Wazuh* combinan el análisis de telemetría de *host* (HIDS) y de red (NIDS) para proveer visibilidad operacional [2].

2.2. Orquestación, Automatización y Respuesta de Seguridad

Las plataformas SOAR se orientan a la interoperabilidad de herramientas aisladas [3]. Un ecosistema SOAR se compone de:

1. **Orquestación:** Integración de herramientas heterogéneas (SIEM, escáneres, inteligencia) mediante interfaces de software, consolidando la información.

2. **Automatización:** Ejecución de tareas programadas (*playbooks*) para reducir la intervención manual.

3. **Respuesta Guiada:** Asistencia al analista mediante interfaces que enriquecen los datos con contexto.

2.3. *Limitaciones de la Evaluación Tradicional*

El estándar CVSS cuantifica la gravedad técnica intrínseca de una falla detectada [4]. A pesar de su adopción, presenta limitaciones operativas: evalúa el vector técnico sin considerar si el activo afectado es un servidor crítico o un equipo de prueba, y carece de un discriminador de explotación activa. Esto contribuye a la sobrecarga cognitiva del analista ante volúmenes masivos de vulnerabilidades etiquetadas como "Críticas".

2.4. *Gestión de Vulnerabilidades Basada en Riesgo*

El paradigma RBVM transiciona la estrategia defensiva desde una mitigación basada en severidad hacia una priorización guiada por el riesgo matemático [5]. El riesgo se modela como una función dependiente de la amenaza real, la severidad intrínseca y el impacto organizacional [6], permitiendo concentrar el esfuerzo operacional en intersecciones críticas.

2.5. *Inteligencia de Amenazas*

La disciplina CTI transforma indicadores técnicos en conocimiento accionable [7]. Para su ingesta se emplean plataformas como MISP [8], las cuales permiten consultar repositorios externos como el catálogo CISA KEV [9], optimizando el cálculo dinámico de riesgo con base en evidencia empírica global.

3. Estado del Arte y Trabajos Relacionados

3.1. *El Problema de la Carga Operacional*

La literatura empírica reciente evidencia las deficiencias del modelo tradicional basado exclusivamente en el SIEM. Tariq et al. [1] diagnostican que la dependencia en reglas estáticas transforma la gestión de eventos en un desafío de sobrecarga cognitiva, desensibilizando a los analistas. Paralelamente, la academia ha demostrado que la métrica CVSS carece de valor defensivo óptimo si no se ajusta al contexto de la red local [4].

3.2. *Evolución hacia la Priorización y el Modelo SSVC*

Para resolver la brecha entre severidad técnica y operación, la industria ha propuesto distintos esquemas. Uno de los enfoques contemporáneos más relevantes es el modelo SSVC desarrollado por la Agencia CISA y la Universidad de Carnegie Mellon. SSVC utiliza árboles de decisión estructurados para categorizar vulnerabilidades en cuatro acciones cualitativas (Track, Attend, Act, Track*), superando la linealidad del CVSS.

Si bien SSVC es el estándar metodológico emergente, su implementación algorítmica en entornos locales automatizados presenta fricciones debido a su naturaleza cualitativa y sus múltiples nodos de decisión manual. Por ello, este trabajo optó por una variante del modelo RBVM utilizando un modelo matemático de suma ponderada. A diferencia de SSVC, la suma ponderada permite obtener un índice numérico continuo (Riesgo Final R_f) que facilita el ordenamiento descendente en interfaces visuales (*Dashboards*) de forma instantánea, simplificando la orquestación en la PoC.

3.3. Aporte y Diferenciación del Proyecto

A diferencia de las soluciones comerciales predictivas opacas (*Machine Learning* cerrado), este proyecto propone un diseño determinístico y trazable [5]. La innovación radica en operacionalizar la teoría mediante un motor que correlaciona ejes de decisión: la severidad técnica, la inteligencia CISA KEV y un multiplicador de criticidad organizacional, operando de forma local para proveer al analista una justificación explícita.

4. Definición del Problema

4.1. Planteamiento del Problema

La efectividad de los SOC depende de su capacidad para responder a incidentes en un tiempo adecuado. Sin embargo, la proliferación de herramientas genera silos de datos. Analizar el impacto de una alerta cruzando plataformas consume en promedio entre 15 y 25 minutos del tiempo de un operador. Dado que gran parte de los eventos reportados son ruido operacional de baja urgencia, la carga de trabajo propicia que un porcentaje significativo de vulnerabilidades sea ignorado, aumentando el riesgo de brechas efectivas. La falta de correlación automatizada entre las vulnerabilidades latentes, la criticidad real de los activos y la inteligencia de amenazas externa impide reducir el esfuerzo de correlación manual.

4.2. Solución Propuesta y Objetivos

Se diseñó e implementó un mecanismo de priorización contextual. El objetivo general es desarrollar una prueba de concepto funcional de un motor de priorización automatizado, inspirado en capacidades de orquestación, que integre telemetría e CTI para optimizar el triaje en un entorno simulado.

Los objetivos específicos son:

1. Definir la arquitectura lógica y los flujos de ingestión de datos estructurados.
2. Diseñar un algoritmo de priorización basado en riesgo combinando CVSS, CISA KEV y criticidad de activos.
3. Desarrollar un motor lógico en Python para la correlación de vulnerabilidades latentes y alertas de telemetría.
4. Implementar una interfaz interactiva en React para la visualización de prioridades.
5. Validar la eficacia del prototipo evaluando analíticamente la reducción de carga cognitiva.

5. Análisis de Requerimientos y Selección Tecnológica

5.1. Requerimientos Operacionales e Infraestructura

Para sostener la ingesta de telemetría y el cálculo algorítmico, el entorno de despliegue se definió bajo las siguientes especificaciones:

Requerimientos de Hardware:. Se requiere un mínimo de 4 vCores, 8 GB de memoria RAM y 100 GB de almacenamiento SSD para procesar la base de datos NoSQL con latencia mínima durante la recolección de eventos.

Software de Entorno Base:. Sistema Operativo Linux (Ubuntu 22.04 LTS o Debian 12) con soporte para Docker. El uso de contenedores garantizó el aislamiento de la base de datos y herramientas auxiliares, mitigando conflictos en el sistema anfitrión.

Arquitectura de Ingesta y Datos Simulados (Mocking):. Para asegurar la reproducibilidad metodológica y el rendimiento de la PoC, se adoptó una arquitectura de ingesta simulada. En lugar de ejecutar peticiones directas y síncronas a las APIs en vivo de OpenVAS o Wazuh, el orquestador se programó para ingerir extractos normalizados en formato JSON almacenados localmente. Esto permitió que el algoritmo se centrara exclusivamente en la lógica de priorización matemática, independizando el análisis de la latencia externa de las plataformas comerciales.

5.2. Justificación Técnica del Stack Tecnológico

La selección de las herramientas se fundamentó en su idoneidad para el procesamiento de datos estructurados:

- **Python (Motor de Orquestación):** Lenguaje seleccionado por su eficiencia en el procesamiento de estructuras de datos (JSON/Diccionarios) y facilidad para modelar algoritmos matemáticos.
- **MongoDB (Capa de Persistencia):** Arquitectura NoSQL orientada a documentos, adecuada para manejar la variabilidad de las alertas sin requerir esquemas rígidos.
- **React.js (Interfaz de Usuario):** Utilizado para desarrollar un *Dashboard* interactivo que consuma la base de datos y presente las vulnerabilidades priorizadas.
- **CISA KEV (Inteligencia de Amenazas):** Catálogo gubernamental empleado para confirmar la explotación activa (I_t). Se descartaron métricas predictivas para evitar basar el riesgo en amenazas meramente teóricas.

6. Diseño de la Solución (Arquitectura)

6.1. Arquitectura Funcional por Capas

La solución se estructuró bajo una arquitectura modular de integración de datos, dividida en distintas fases lógicas (ver anexos para esquemas detallados):

- **Capa de Contexto (Archivos Locales):** Extractos JSON simulando el comportamiento de sensores. Proveen las alertas operacionales (Wazuh), las vulnerabilidades latentes con sus respectivos puntajes CVSS (OpenVAS) y el inventario de criticidad de los activos.
- **Capa de Inteligencia (CTI):** Referencia cruzada con el catálogo CISA KEV para validar si el CVE reportado posee evidencia histórica de explotación global.
- **Capa Lógica (Motor Python):** Algoritmo central de ejecución nativa que correlaciona las variables anteriores y aplica el modelo matemático para cada vulnerabilidad.
- **Capa de Presentación:** Los resultados consolidados se insertan en MongoDB y son expuestos mediante una interfaz web reactiva.

6.2. *Modelamiento Matemático de la Fórmula de Riesgo*

Basado en los lineamientos de la norma **ISO/IEC 27005:2022**, el riesgo considera las consecuencias de un evento y la probabilidad de su ocurrencia. Para proveer contexto organizacional a la métrica CVSS, se diseñó un **modelo de suma ponderada**.

El modelo calcula un Multiplicador Dinámico (M_{din}) agrupando tres factores analíticos:

$$M_{din} = (E_o \times 0,45) + (C_a \times 0,35) + (I_t \times 0,20) \tag{1}$$

El Riesgo Final (R_f) se obtiene multiplicando la severidad base por este factor dinámico. El resultado se divide por $1,2$, el cual corresponde al **límite máximo analítico** de la ecuación ($1,2 \times (0,45 + 0,35 + 0,20) = 1,2$). Esta división garantiza que el puntaje final jamás exceda la escala máxima de $10,0$.

Es fundamental destacar que este modelo actúa matemáticamente como un **atenuador**. Si una vulnerabilidad carece de contexto de ataque real o afecta a un equipo de baja criticidad, el multiplicador será inferior a $1,2$, degradando el puntaje CVSS original para mitigar el ruido operacional.

$$R_f = \frac{\text{CVSS} \times M_{din}}{1,2} \tag{2}$$

6.3. *Diccionario de Variables y Asignación de Pesos*

Los pesos del modelo fueron definidos mediante revisión de literatura y criterio experto heurístico, utilizando factores de escala ($0,5$ a $1,2$). Cabe señalar que estos valores no fueron validados estadísticamente con analistas en entornos productivos durante esta investigación.

- **Severidad Técnica Base (CVSS $\in [0,0,\ 10,0]$):** Radicando técnico inmutable.
- **Evidencia Operativa Local (E_o - Peso: 45 %):** Posee el peso mayoritario al representar actividad inminente en la red local. Sus niveles son:
  - $0,5$ (Limpio): Ausencia de alertas concurrentes en el activo.
  - $0,8$ (Sospechoso): Alertas de nivel bajo o medio en el segmento.
  - $1,2$ (Comprometido): Alertas críticas indicativas de ataque activo.
- **Criticidad del Activo (C_a - Peso: 35 %):** Impacto organizacional del equipo.
  - $0,5$ (Baja): Entornos de prueba aislados.
  - $0,8$ (Media): Estaciones de trabajo estándar.
  - $1,0$ (Alta): Servidores en producción.
  - $1,2$ (Crítica): Bases de datos centrales expuestas.
- **Inteligencia de Amenazas (I_t - Peso: 20 %):** Contexto de explotación global.
  - $0,5$ (Teórica): Sin evidencia pública de explotación activa.
  - $0,8$ (PoC): Existencia de prueba de concepto pública.
  - $1,2$ (Activa): Presente en el catálogo CISA KEV.

6.4. Análisis de Sensibilidad de la Fórmula

Para comprobar que el modelo respeta la lógica de atenuación, se evaluó la Ecuación 2 en escenarios extremos:

1. **Escenario A (Límite Máximo):** Vulnerabilidad Crítica ($CVSS = 10,0$) en servidor Crítico ($C_a = 1,2$), en CISA ($I_t = 1,2$) y con ataque detectado ($E_o = 1,2$).
   - $R_f = (10,0 \times 1,2)/1,2 =$ **10,0**
   - *Análisis:* El modelo escala sin desbordar el rango máximo permitido.
2. **Escenario B (Ruido Operacional Crítico):** Vulnerabilidad Crítica ($CVSS = 10,0$) en un equipo de pruebas ($C_a = 0,5$), sin inteligencia ($I_t = 0,5$) y sin telemetría de ataque ($E_o = 0,5$).
   - $M_{din} = (0,5 \times 0,45) + (0,5 \times 0,35) + (0,5 \times 0,20) = 0,5$
   - $R_f = (10,0 \times 0,5)/1,2 = 4,166 \approx$ **4,17**
   - *Análisis:* El modelo cumple su rol atenuador. Una vulnerabilidad de puntaje $10,0$ se degrada a $4,17$, mitigando el impacto visual de alertas de baja urgencia táctica.
3. **Escenario C (Peligro Activo de Severidad Media):** Vulnerabilidad Media ($CVSS = 5,0$) en un servidor estándar ($C_a = 1,0$), enlistada en CISA ($I_t = 1,2$) y con ataque en curso ($E_o = 1,2$).
   - $M_{din} = (1,2 \times 0,45) + (1,0 \times 0,35) + (1,2 \times 0,20) = 1,13$
   - $R_f = (5,0 \times 1,13)/1,2 = 4,708 \approx$ **4,71**
   - *Análisis:* La intrusión real $(4,71)$ supera matemáticamente a la vulnerabilidad teórica del Escenario B $(4,17)$, confirmando la jerarquía buscada.

7. Validación de la Solución

7.1. Hipótesis de Investigación

Para evaluar el impacto del sistema propuesto frente a la problemática de la carga operacional en un SOC, se plantearon las siguientes hipótesis formales:

- **Hipótesis Nula (H_0):** El modelo de triaje dinámico RBVM propuesto no presenta una mejora en la priorización de vulnerabilidades explotables (Precision@K) respecto al triaje estático tradicional basado únicamente en el puntaje CVSS.
- **Hipótesis Alternativa (H_1):** El modelo RBVM incrementa el Precision@K, ubicando las vulnerabilidades con intrusiones confirmadas en los primeros percentiles de la interfaz, reduciendo el esfuerzo de correlación analítica.

7.2. Diseño Experimental y Líneas Base

El estudio cuantitativo contrastó la eficacia del motor en un entorno de laboratorio frente a dos escenarios de control (Líneas Base) que representan enfoques de la industria:

- **Línea Base 1 (LB1 - Triaje Estático):** Las vulnerabilidades se ordenaron de mayor a menor únicamente por su severidad técnica estructural (CVSS Base).
- **Línea Base 2 (LB2 - Triaje Contextualizado):** Modelo intermedio que cruzó la severidad técnica con el valor del activo afectado ($CVSS \times C_a$), pero que careció de validación de inteligencia en tiempo real (I_t y E_o).

7.3. Dataset y Metodología (Ground Truth Independiente)

Para garantizar la reproducibilidad y evitar un riesgo de evaluación circular, el *Ground Truth* o verdad fundamental de los incidentes se definió utilizando un estándar cualitativo e independiente. Se empleó el árbol de decisiones del modelo SSVC.

El procedimiento constó de las siguientes fases:

1. **Evaluación SSVC (Etiquetado Independiente):** Antes de ejecutar la plataforma SOAR, las vulnerabilidades fueron evaluadas manualmente a través de los nodos lógicos de SSVC (Explotación, Impacto en la Misión, Automatización e Impacto Técnico). Debido a que los activos de prueba ($C_a = 0,5$) poseen un Impacto de Misión catalogado como *Degraded*, el árbol SSVC determinó que las vulnerabilidades Críticas (CVSS 10,0) en esos equipos solo requerían la acción *Track*.
2. **Fuente de Telemetría Simulada:** Se generó un lote inyectado de 50 eventos operacionales en formato JSON, replicando vectores de ataque.
3. **Ejecución del Orquestador:** El motor lógico ingirió los eventos y las vulnerabilidades latentes, aplicando el algoritmo matemático RBVM de manera aislada a las etiquetas generadas por SSVC.
4. **Comparación Analítica:** Se contrastó el Top 5 ordenado matemáticamente por el motor RBVM contra las etiquetas cualitativas *Act* generadas por SSVC.

7.4. Métricas de Evaluación

Se utilizaron dos métricas principales para cuantificar el desempeño de la PoC:

- **Precision@K (Precisión en el Top-K):** Evaluó la calidad de los primeros K resultados ($K = 5$ y $K = 10$) en la pantalla del analista. El éxito se midió comparando el ordenamiento del motor contra la etiqueta de urgencia independiente (SSVC).
- **Reducción del Esfuerzo de Correlación Manual:** Se estimó analíticamente el impacto operativo comparando la carga de tareas de investigación manual frente al procesamiento automatizado.

7.5. *Aplicación y Análisis de Pruebas Unitarias*

Para verificar la integridad matemática del motor, se implementó una batería de pruebas unitarias (*Unit Testing*) utilizando la librería `unittest` de Python.

La automatización evaluó cuatro dimensiones lógicas:

- **Control de Saturación (R_f):** Se inyectó el escenario más crítico posible ($CVSS = 10,0$, $E_o = 1,2$, $C_a = 1,2$, $I_t = 1,2$). La aserción validó que la normalización matemática lograra que el resultado no excediera el límite estricto de $10,0$.
- **Atenuación de Ruido Operacional:** Se comprobó la degradación controlada del puntaje original al inyectar un evento de bajo impacto ($E_o = 0,5$) sobre un equipo de prueba ($C_a = 0,5$).
- **Traducción de Telemetría:** Se validó que el *parser* asigne el peso correcto según el nivel de alerta original (ej. Nivel ≥ 10 asigna $1,2$).
- **Integración CTI:** Se verificó que la lógica de consulta asigne la ponderación máxima únicamente a los CVE reportados como explotados activamente.

Los resultados evidenciaron una correcta ejecución, procesando los escenarios sin generar excepciones lógicas ni desviaciones en los redondeos decimales.

7.6. *Aplicación y Análisis de Pruebas de Integración*

Para asegurar la resiliencia del orquestador, se diseñó una batería de pruebas de integración (*handshake*) orientada a validar la correcta extracción y persistencia de datos.

Tabla 2: Matriz de Pruebas de Integración de Componentes.

Componente	**Criterio de Validación**
Persistencia (MongoDB)	*Ping* de conexión exitoso al demonio local (TCP 27017) y escritura de prueba de un documento JSON.
Contexto Estático (OpenVAS e Inventario)	Lectura exitosa de los archivos físicos locales de escaneo (`openvas_real.json`) y matriz de criticidad (`inventario.json`).
Telemetría Dinámica (Wazuh)	Acceso y lectura exitosa del *log* de eventos simulado (`alerts.json`) para extraer la IP y nivel de regla.

7.7. *Validación de la Hipótesis y Comparativa de Resultados*

Para evaluar la Hipótesis de Investigación definida en la Sección 7.1, se contrastó el *Ground Truth* (determinado por las etiquetas *Act* del árbol SSVC en la Tabla E.1) con los resultados empíricos expuestos en la interfaz del orquestador bajo los tres esquemas de priorización.

El análisis del comportamiento de las tres líneas base revela lo siguiente:

- **Línea Base 1 (Solo CVSS):** El ordenamiento puramente técnico reflejó problemas de fatiga de alertas. El CVE-2020-1472 (Zerologon) logró posicionarse en el lugar 4 con un puntaje de $10,00$ a pesar de estar en un entorno de pruebas aislado (etiquetado como *Track* por SSVC). Esta intromisión de ruido operacional redujo el Precision@5 al 80 %.

- **Línea Base 2 (CVSS $\times C_a$):** Si bien alcanzó un Precision@5 del 100 % al desplazar la vulnerabilidad de pruebas, lo hizo sacrificando la utilidad operativa. El multiplicador de criticidad provocó una colisión de puntajes: cinco vulnerabilidades empataron con un puntaje de 10, 00. Esto limitó la capacidad del analista para priorizar dentro de ese subgrupo crítico, generando un escenario que no reduce el esfuerzo analítico.

- **Modelo RBVM (Propuesto):** El motor SOAR logró un Precision@5 del 100 % de forma heurística. La fórmula restauró la granularidad matemática (diferenciando las amenazas de 10, 00, 9, 80 y 9, 23 en el Top 5) gracias a la evaluación de la telemetría del SIEM y la inteligencia CTI. En la métrica Precision@10 (70 %), el modelo evidenció su balance: la ecuación determinó que un CVSS 10, 00 en un equipo de pruebas bajo ataque real (R_f = 7, 96) representa un riesgo marginalmente superior a un CVSS 7, 8 en un servidor web (R_f = 7, 34), desplazando lógicamente incidentes *Act* de baja severidad base hacia el final del Top 10.

Conclusión sobre la Hipótesis de Investigación:
Los datos estructurados confirman que el modelo estático tradicional (LB1) permite el paso de falsas urgencias, al carecer de filtrado de contexto. Por el contrario, la orquestación RBVM incrementó el Precision@5 del 80 % al 100 %, aislando de manera precisa en la cima de la cola de trabajo a las vulnerabilidades catalogadas como emergencias organizacionales (*Act*) y minimizando los empates matemáticos. Con base en esta evidencia empírica, **se rechaza la Hipótesis Nula (H0) y se acepta la Hipótesis Alternativa (H1)**. Se valida que el modelo de triaje dinámico propuesto incrementa la precisión y reduce el esfuerzo de correlación analítica requerido por los operadores del SOC.

La validación empírica de estos resultados numéricos se materializa en la interfaz gráfica del orquestador (detallada en el Anexo Anexo F). Al analizar la Figura F.1 (Línea Base 1), el ordenamiento puramente técnico evidencia el problema fundamental de la fatiga de alertas. Al aplicar el contexto del inventario en la Figura F.2 (Línea Base 2), se logra una atenuación parcial del ruido. Finalmente, la vista del Modelo RBVM (Figura F.3) respalda la eficacia del algoritmo, aislando de manera precisa el Top 5 de emergencias reales y replicando computacionalmente la decisión de priorización *Act* del estándar SSVC.

7.8. Evaluación Analítica de Carga de Trabajo

Para validar el impacto operativo frente al usuario, se modeló analíticamente la carga cognitiva requerida para priorizar vulnerabilidades.

En un escenario tradicional, un operador realiza al menos cinco transacciones manuales por incidente: leer la alerta, buscar la IP en el escáner, cotejar el rol del equipo y validar indicadores. Estimaciones de la industria sugieren que este proceso consume un promedio de 9 a 15 minutos por alerta [10]. Para correlacionar un lote de 50 eventos operacionales en bruto contra reportes de vulnerabilidades, el triaje manual exigiría aproximadamente entre 5 y 7,5 horas de procesamiento ininterrumpido.

Al someter los datos al motor SOAR, la plataforma ejecutó la deduplicación de eventos y las evaluaciones multicriterio de forma automatizada mediante integraciones locales. El cálculo analítico consolidó la información en la interfaz gráfica en tiempos computacionales mínimos. Esto sugiere una reducción significativa del esfuerzo de correlación manual, transformando la lectura de *logs* dispersos en una toma de decisiones fundamentada en riesgo.

Tabla 3: Comparativa de Resultados de Priorización frente a Líneas Base.

Métrica	**LB1 (Solo CVSS)**	**LB2 (CVSS $\times C_a$)**	**Modelo RBVM (Final)**
Precision@5	80 % (4 ACT). El CVE de pruebas ocupó la posición 4 de la pantalla.	100 % (5 ACT). Colisión de puntajes (cinco empates críticos en $10,00$).	**100 % (5 ACT).** Granularidad restaurada sin empates en el Top 5.
Precision@10	60 % (6 ACT). Se priorizaron falsas urgencias de entornos aislados.	90 % (9 ACT). Los CVE de impacto medio en producción subieron agresivamente.	**70 % (7 ACT).** Balance matemático: CVSS críticos en pruebas superaron marginalmente a CVSS bajos en producción.
Supresión de Ruido	Nula. Bloqueó el flujo visual del Top 5 táctico.	Ciega. Ignoró la telemetría de ataque cayendo matemáticamente de forma abrupta.	**Equilibrada.** Desplazamiento controlado al fondo de la cola visual ($R_f = 7,96$) reteniendo el contexto de ataque activo.

8. Limitaciones

El diseño e implementación de esta Prueba de Concepto contempló las siguientes restricciones:

- **Correlación basada en IP (*Host*):** El motor lógico diseñado eleva el riesgo de todas las vulnerabilidades de un activo si este recibe una alerta de intrusión, aunque el evento no tenga relación directa con el CVE específico. Se identificó la necesidad de un nivel de granularidad mayor para mapear puertos y servicios afectados.
- **Validación de Pesos Matemáticos:** Los factores ponderados de la fórmula ($0,45$ para E_o; $0,35$ para C_a; $0,20$ para I_t) fueron establecidos heurísticamente. No se realizaron pruebas experimentales con analistas SOC en entornos de producción para validar estadísticamente su precisión frente al juicio humano.
- **Infraestructura y *Mocking*:** Para asegurar la reproducibilidad sin penalizar el *hardware* local, el sistema operó mediante lectura de archivos locales (*mocking* de JSON) en lugar de conexiones en tiempo real hacia las APIs nativas de Wazuh y OpenVAS.
- **Alcance Exclusivo de Priorización:** Esta plataforma operó como una herramienta de orquestación analítica orientada al triaje. No se contempló el desarrollo de *playbooks* para la respuesta automatizada intrusiva en la red (ej. bloqueo de direcciones IP en el *firewall*).

9. Conclusiones y Trabajo Futuro

El presente trabajo logró el diseño e implementación de un mecanismo de priorización contextual centralizado, cumpliendo con los objetivos propuestos. La arquitectura evidenció la viabilidad de unificar fuentes de datos heterogéneas —telemetría operacional, escaneos estructurales e inteligencia externa— en un flujo de trabajo consolidado, mitigando la fragmentación intrínseca de los SOC.

A través del modelo matemático RBVM implementado, se superó la limitación de depender de métricas estáticas. Los resultados sugieren que el uso de un multiplicador dinámico actúa eficazmente como un atenuador de ruido operacional, degradando el nivel de urgencia de vulnerabilidades en entornos no críticos, y elevando de forma determinística aquellas que presentan explotación activa comprobada. La evaluación analítica respaldó la eficacia del prototipo, indicando una reducción del esfuerzo de correlación manual requerido por un analista.

Como trabajo futuro, el desarrollo de esta plataforma puede extenderse en tres dimensiones:

1. **Granularidad de Detección:** Desarrollar un mapeo enriquecido que correlacione específicamente el puerto o servicio del evento alertado en el SIEM con el vector de red del CVE reportado, eludiendo la elevación de riesgo colateral a nivel de IP.
2. **Conectividad en Tiempo Real:** Reemplazar la ingesta mediante *mocking* local por integraciones bidireccionales síncronas hacia APIs productivas.
3. **Automatización de Respuesta:** Evolucionar el motor de priorización hacia una plataforma SOAR integral mediante el desarrollo y orquestación de *playbooks* que permitan ejecutar acciones de contención automatizadas directamente desde el *Dashboard*.

Agradecimientos

El autor agradece a la Escuela de Ingeniería Informática de la Universidad de Valparaíso y al profesor guía Cristian Carrión por el apoyo brindado en la formulación de este proyecto.

Aspectos éticos

Se confirma que el desarrollo de este trabajo se enmarcó en un entorno de laboratorio aislado utilizando bases de datos públicas y datos estructurados controlados. No se comprometió información sensible ni se afectaron infraestructuras productivas reales, alineándose con las normativas éticas de ciberseguridad corporativa.

Declaración de autoría

Jorge Eduardo Álvarez Alarcón: Conceptualización, Metodología, Software, Validación, Investigación, Redacción de borrador original y edición.

Declaración de intereses

El autor declara no tener conflictos de intereses financieros, personales ni institucionales que pudieran influir en los resultados o interpretaciones del presente trabajo.

Referencias

[1] S. Tariq, M. B. Chhetri, S. Nepal, C. Paris, Alert fatigue in security operations centres: Research challenges and opportunities, ACM Computing Surveys (2025). doi:10.1145/3723158.

[2] M. Amami, A. Charfeddine, Exploration of open source siem tools and deployment of an appropriate wazuh-based solution for strengthening cyberdefense, in: IEEE Xplore, 2024, pp. 1–8. doi:10.1109/10708054/10708053/10708476.

[3] B. D. Jaiswal, Designing scalable software automation frameworks for cybersecurity threat detection and response, International Journal of Scientific Research and Management (IJSRM) 13 (02) (2025) 1958–1980. doi:10.18535/ijsrm/v13i02.ec03.

[4] P. Pakshad, et al., Vulnerability scoring metric of cvss needs to be adjusted per each product: our analysis on linux and apache, Information Security Journal: A Global Perspective (2025).

[5] M. Rieppi, Vulnerability management: A risk-based optimization, Master's thesis, Università degli Studi di Padova, Dipartimento di Matematica "Tullio Levi-Civita"(2025). URL https://hdl.handle.net/20.500.12608/101996

[6] K. G. Agyei, Explainable risk-based vulnerability prioritization in hybrid cloud: Integrating cvss, epss, and cisa kev with asset criticality signals, World Journal of Advanced Research and Reviews 30 (1) (2026) 2044–2052. doi:10.30574/wjarr.2026.30.1.1006.

[7] A. Abakar, et al., A cyber threat intelligence model using misp and machine learning in a soc environment, International Journal of Advanced and Applied SciencesOpen Access (2025).
URL https://www.science-gate.com/IJAAS/Articles/2025/2025-12-11/1021833ijaas202511001.pdf

[8] R. Kits, et al., What's in a cyber threat intelligence sharing platform?: A mixed-methods user experience investigation of misp, in: Proceedings of the ACM, 2021, pp. 1–12. doi:10.1145/3485832.3488030.

[9] runZero, Kevology: An analysis of exploits, scores, and timelines on the cisa kev, Tech. rep., runZero (2024).
URL https://www.runzero.com/uploads/documents/reports/KEVology-An-analysis-of-exploits-scores-and-timelines-on-the-CISA-KEV.pdf

[10] Palo Alto Networks, 2023 state of cloud-native security report, Industry report, Palo Alto Networks (2023).

Anexo A. Diagramas de Ingeniería de Software

Anexo A.1. Arquitectura de Alto Nivel y Despliegue

El siguiente diagrama detalla la topología de la solución propuesta, ilustrando la separación funcional entre la capa de captura, la capa de orquestación y la capa de presentación.

Anexo A.2. Diagrama de Paquetes (Diseño Lógico)

La estructuración del código fuente se dividió en módulos independientes para facilitar la escalabilidad del orquestador.

Anexo A.3. Catálogo de Requerimientos y Casos de Uso

ID	Descripción del Requerimiento	Tipo
RF-01	El motor lógico debe ingerir las alertas operacionales concurrentes simuladas para calcular el factor E_o.	Funcional
RF-02	El sistema debe consultar la base de inteligencia para verificar la presencia de un CVE en el catálogo CISA KEV (I_t).	Funcional
RF-03	El sistema debe calcular el índice dinámico RBVM combinando severidad, inteligencia, telemetría y criticidad.	Funcional
RF-04	La interfaz (Dashboard) debe presentar las vulnerabilidades ordenadas de forma descendente según el Riesgo Final (R_f).	Funcional
RNF-01	**Soberanía de Datos:** El procesamiento de telemetría debe realizarse de forma local.	No Funcional (Seguridad)
RNF-02	**Tecnología:** El motor de orquestación central debe ser desarrollado en lenguaje Python 3.x.	No Funcional (Arquitectura)

Tabla A.1: Catálogo de requerimientos principales de la PoC.

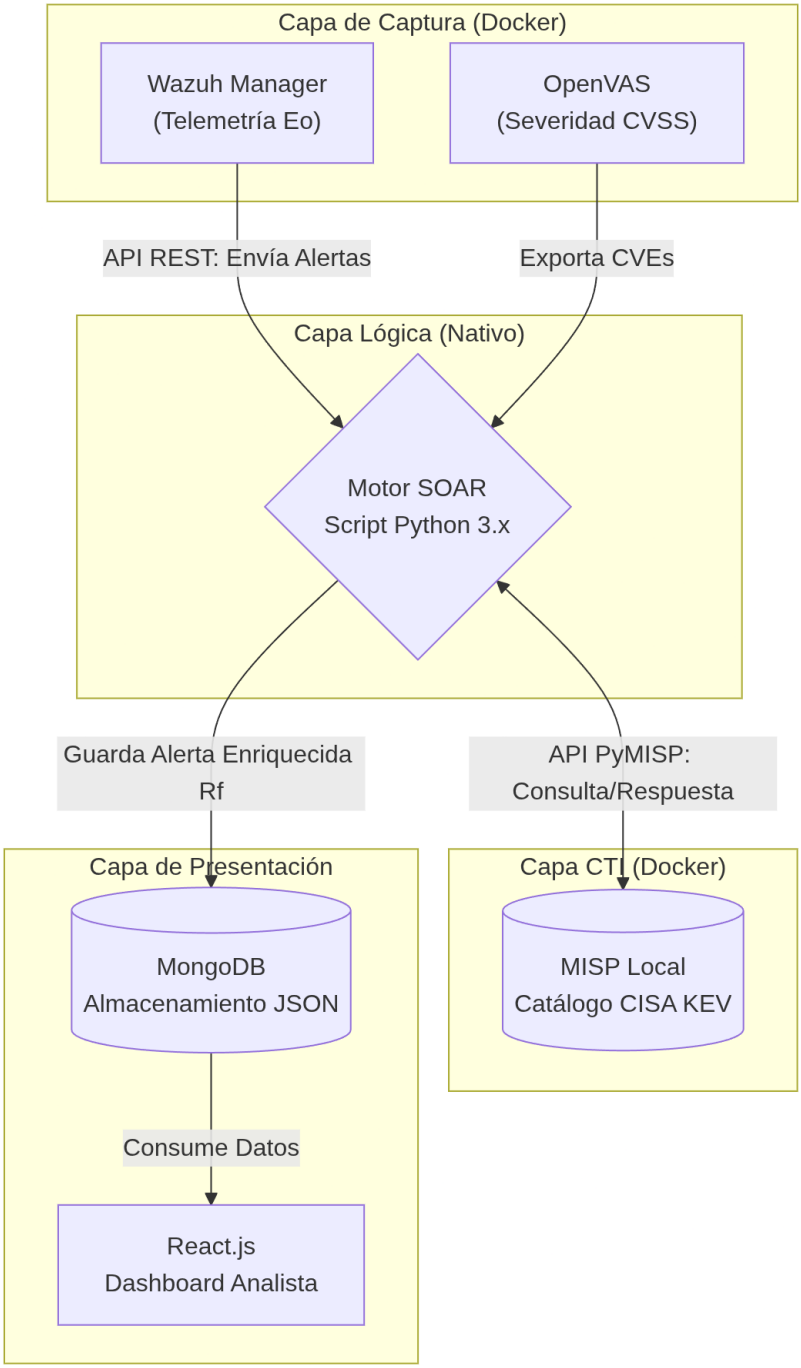


Figura A.1: Esquema de arquitectura de alto nivel de la PoC.

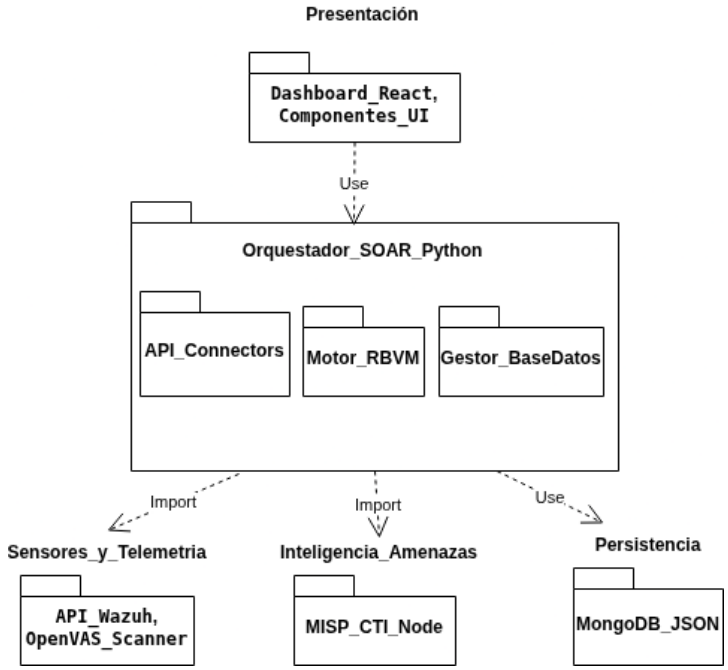


Figura A.2: Diagrama de Paquetes que ilustra las dependencias internas del Motor Python.

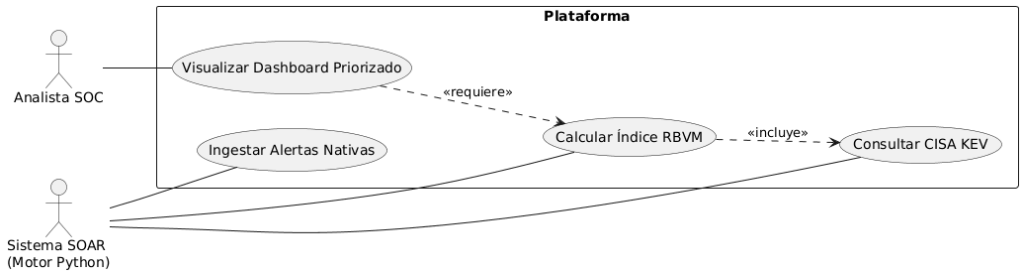


Figura A.3: Diagrama de Casos de Uso: Interacción entre el analista y el orquestador.

Anexo A.4. Diagramas de Secuencia y Estado

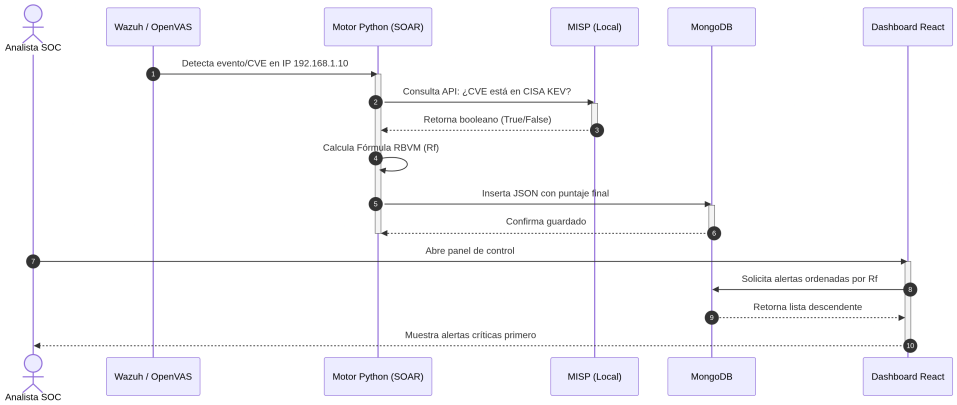


Figura A.4: Diagrama de Secuencia: Flujo automatizado de triaje y priorización RBVM.

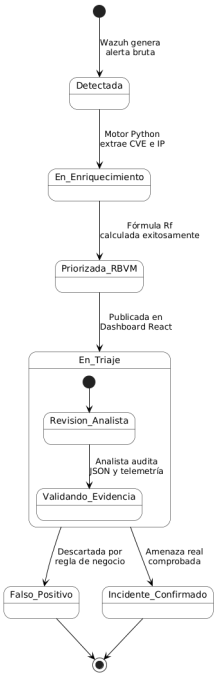


Figura A.5: Diagrama de Estado: Ciclo de vida transaccional de un incidente.

Anexo B. Modelo de Datos y Especificaciones JSON

La estructura prescinde de un modelo Entidad-Relación tradicional en favor de un esquema de documentos JSON anidados almacenados en MongoDB. A continuación, se presenta el esquema resultante de la correlación automatizada:

```
{
  "_id": "100202-192.168.1.10",
  "timestamp_procesamiento": "2026-06-21T14:22:10Z",
  "activo": {
    "ip": "192.168.1.10",
    "hostname": "SRV-BD-FINANZAS",
    "criticidad_ca": 1.2
  },
  "integracion_openvas": {
    "cve_id": "CVE-2021-44228",
    "score_base_cvss": 10.0
  },
  "telemetria_wazuh": {
    "ataque_detectado": true,
    "nivel_regla": 12,
    "factor_eo": 1.2
  },
  "calculo_rbvm": {
    "multiplicador_dinamico": 1.2,
    "riesgo_final_rf": 10.0,
    "estado": "En_Triaje"
  }
}
```

Anexo C. Diseño Algorítmico del Motor de Priorización

El siguiente pseudocódigo define la lógica del *script* en Python encargado de orquestar los datos de las herramientas y aplicar la fórmula de atenuación de riesgo diseñada para este proyecto:

```
INICIO FUNCION procesar_triaje_rbvm(evento_wazuh, archivo_openvas, inventario)
    // 1. Extraer identificadores base
    ip_afectada = evento_wazuh.agente.ip
    nivel_wazuh = evento_wazuh.regla.nivel

    // 2. Correlacionar con JSON OpenVAS (Contexto Estructural)
    cve, cvss = buscar_vulnerabilidad_en_openvas(ip_afectada, archivo_openvas)
    SI cve ES NULO ENTONCES RETORNAR // Falla sin contexto técnico

    // 3. Extraer Variables Ponderadas
    ca = extraer_criticidad_inventario(ip_afectada, inventario)
    it = consultar_cti_cisa(cve) // Retorna 1.2 (Activo) o 0.5 (Teórico)

    // Evaluar Evidencia Operativa (Reglas >= 10 son intrusiones críticas)
    SI nivel_wazuh >= 10 ENTONCES eo = 1.2 SINO eo = 0.5

    // 4. Calcular Riesgo Final (Modelo Suma Ponderada)
    m_din = (eo * 0.45) + (ca * 0.35) + (it * 0.20)
    riesgo_final = (cvss * m_din) / 1.2

    // 5. Guardar en Base de Datos NoSQL
    JSON_enriquecido = construir_documento(ip_afectada, cvss, m_din, riesgo_final)
    mongodb.insertar(JSON_enriquecido)

    RETORNAR REDONDEAR(riesgo_final, 2)
FIN FUNCION
```

Anexo D. Definición de Integraciones y Reglas de Orquestación

El flujo operativo de la plataforma definió las siguientes reglas y conectores lógicos para la orquestación, simuladas localmente para la investigación:

- **Ingesta de Telemetría (SIEM):** Basada en la estructura del *endpoint* de Wazuh, se diseñó la lógica para iterar sobre eventos JSON, identificando la variable `rule.level` para determinar el factor de Evidencia Operativa (E_o).
- **Análisis de Vulnerabilidades:** Basado en los reportes de OpenVAS, se estableció una regla de lectura para asociar las direcciones IPv4 con sus vulnerabilidades estáticas (CVSS Base), constituyendo el insumo principal de riesgo latente.
- **Lógica de Inteligencia de Amenazas:** Se definió una regla de consulta contra una base de datos CTI que evalúa la presencia del CVE. Si el identificador cruza afirmativamente con el catálogo de explotación global, se aplica un factor agravante al cálculo final (I_t).

Anexo E. Reproducibilidad y Dataset Experimental

Para favorecer la reproducibilidad académica de esta Prueba de Concepto, los *scripts* del motor lógico en Python y los archivos JSON con la estructura de las bases de datos simuladas se versionaron y dispusieron para revisión.

- **Repositorio de Código Fuente:** `https://github.com/jalvarez-uv/soar-rbvm-poc`
- **Dataset Estructural:** El archivo `inventario.json` consolidó los 4 activos de prueba parametrizados con los siguientes niveles de criticidad organizacional (C_a):
  - `192.168.1.10` - Servidor de Producción / Base de Datos ($C_a = 1,2$)
  - `192.168.1.20` - Servidor Web / Aplicación ($C_a = 1,0$)
  - `192.168.1.30` - Estación de Trabajo Estándar ($C_a = 0,8$)
  - `192.168.1.50` - Entorno de Calidad / Pruebas ($C_a = 0,5$)
- **Dataset de Eventos (*Ground Truth*):** El archivo `alerts.json` agrupó el lote inyectado de 50 eventos operacionales. Las etiquetas de urgencia de mitigación se asignaron utilizando los árboles cualitativos del estándar SSVC, lo que permitió evaluar la priorización de la plataforma sin incurrir en validaciones lógicas circulares.

Anexo E.1. Fundamentos y Aplicación del Modelo SSVC

Para aislar la evaluación del motor RBVM, se utilizó el estándar SSVC en su perfil de Defensor (*Deployer*). A diferencia del CVSS, que genera un puntaje numérico, SSVC empleó un árbol de decisiones cualitativo compuesto por cuatro nodos que desembocaron en una acción de mitigación.

Los cuatro nodos evaluados para cada vulnerabilidad fueron:

1. **Explotación (*Exploitation*):** ¿Existió evidencia de explotación? Se categorizó en *None* (Ninguna), *PoC* (Prueba de concepto pública) o *Active* (Explotación comprobada).
2. **Impacto Técnico (*Technical Impact*):** Grado de control obtenido por el atacante. Se dividió en *Partial* (Control limitado) o *Total* (Compromiso absoluto del sistema).
3. **Automatizable (*Automatable*):** ¿Pudo un atacante explotar la vulnerabilidad de forma masiva y autónoma? Se respondió con *Yes* o *No*.
4. **Impacto en la Misión (*Mission Impact*):** Efecto organizacional ante el compromiso del activo. Los servidores de Producción y Web poseyeron impacto *Crippled* (Devastador), mientras que las estaciones de trabajo y entornos de prueba se catalogaron como *Degraded* (Degradado).

Las respuestas a estos nodos dirigieron a una de tres acciones finales: ***Track*** (Rastrear y mitigar en ciclo normal), ***Attend*** (Atender y agendar parcheo a corto plazo), o ***Act*** (Emergencia crítica, mitigar de inmediato).

Evaluación Manual de las Vulnerabilidades

La Tabla E.1 presenta el recorrido analítico de las 10 vulnerabilidades principales que lideraron la cola de trabajo, evaluadas a través del árbol SSVC. Este etiquetado conformó el *Ground Truth* del experimento.

Tabla E.1: Evaluación paso a paso de los 10 CVEs principales en el árbol de decisiones SSVC.

IP Base	CVE	Explot.	Auto	Imp. Téc	Imp. Misión	Decisión
.10 (Prod)	CVE-2021-44228	Active	Yes	Total	Crippled	**ACT**
.10 (Prod)	CVE-2020-0796	Active	Yes	Total	Crippled	**ACT**
.10 (Prod)	CVE-2017-5638	Active	No	Total	Crippled	**ACT**
.10 (Prod)	CVE-2019-11043	Active	No	Total	Crippled	**ACT**
.10 (Prod)	CVE-2022-22965	Active	No	Partial	Crippled	**ACT**
.20 (Web)	CVE-2020-1938	Active	No	Total	Crippled	**ACT**
.30 (Admin)	CVE-2019-0708	Active	Yes	Total	Degraded	**Attend**
.50 (Prueba)	CVE-2020-1472	Active	No	Total	Degraded	**Track**
.30 (Admin)	CVE-2021-34527	Active	No	Total	Degraded	**Attend**
.20 (Web)	CVE-2019-0211	Active	No	Partial	Crippled	**ACT**

Anexo F. Vistas del Orquestador y Comparativa Empírica

Para la fase de interacción visual, se optó por un prototipado funcional en código utilizando la biblioteca **React.js**. Esta decisión permitió que la interfaz gráfica actuara como una vista dinámica capaz de consumir y renderizar de forma asíncrona los resultados del algoritmo en Python almacenados en la base de datos NoSQL.

El diseño visual adoptó un esquema *Dark Mode* nativo, implementando una semántica de colores para guiar la atención del operador: riesgo crítico ($R_f \geq 9,0$) en tonos rojos, y riesgo atenuado ($R_f \leq 4,0$) en tonos azules. Adicionalmente, se desarrolló una funcionalidad de alternancia de vistas que permitió al analista comparar empíricamente el comportamiento del ordenamiento bajo las líneas base frente al modelo propuesto. A continuación, se exponen las capturas del sistema en sus tres estados de evaluación:

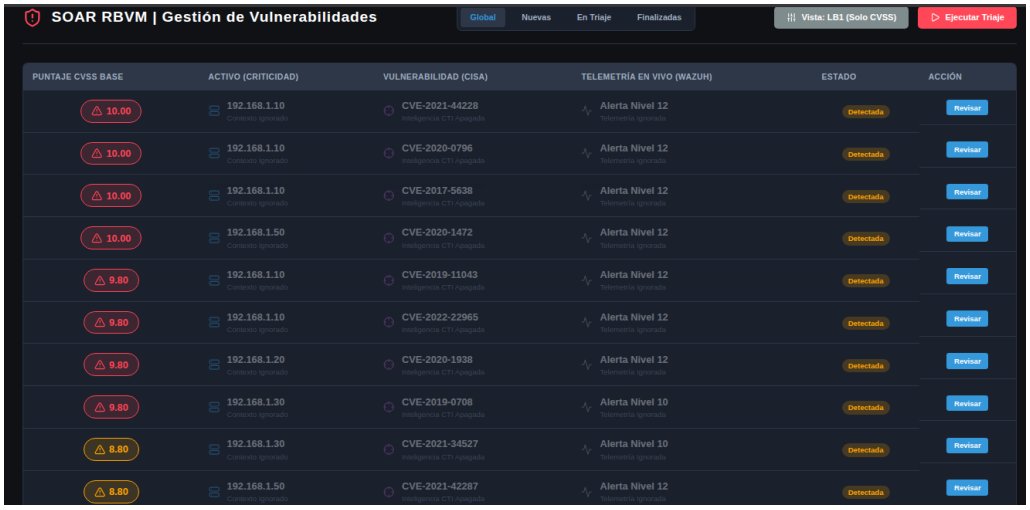


Figura F.1: Vista LB1 (Solo CVSS Base): Se observó la carencia de contexto; vulnerabilidades de entornos de prueba compartieron jerarquía visual con los servidores de producción.

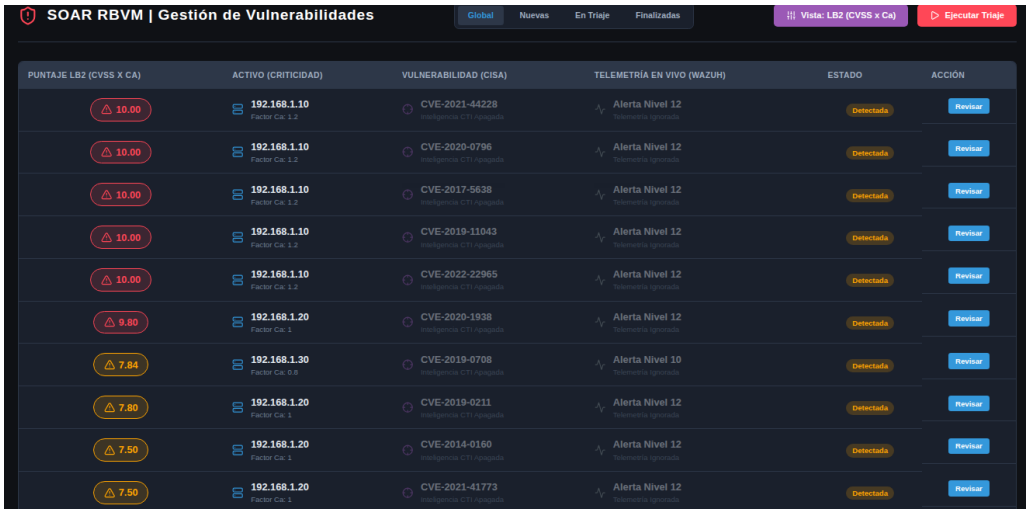


Figura F.2: Vista LB2 (CVSS $\times C_a$): El sistema iluminó el Factor C_a. Se atenuaron los activos de prueba, pero persistieron en los primeros lugares alertas carentes de telemetría de ataque operacional.

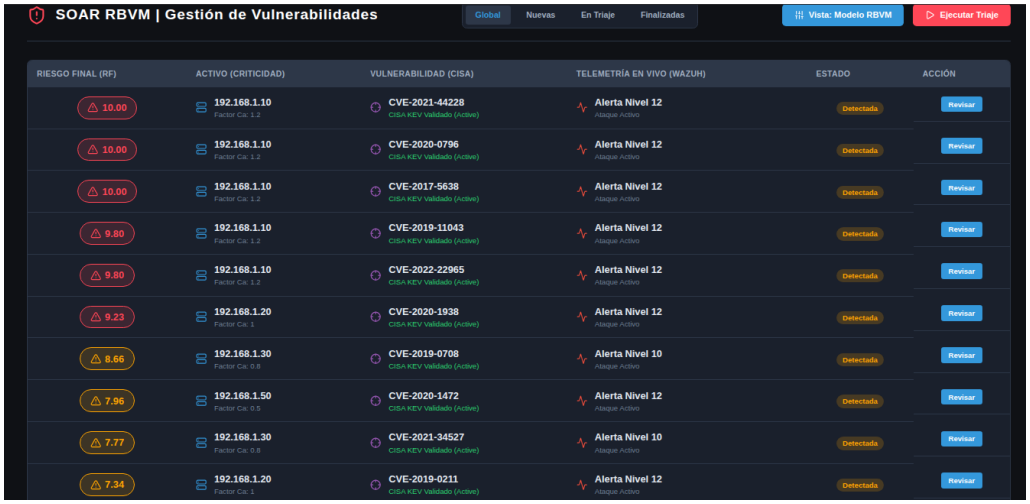


Figura F.3: Vista Modelo RBVM (Final): Integración de los ejes de decisión. Se encendieron las variables de CTI y Telemetría. El orquestador posicionó en el Top 5 a las amenazas con explotación confirmada en activos organizacionales críticos.